Beyond Recall: Recovery Contracts for Agent Memory Mutations

Aditya Mantri*
Independent Researcher

October 29, 2026

Abstract

A memory update can recover its expected facts while duplicating records, repeating work, or overwriting a newer writer. We study these distinctions through a synthetic contract that independently varies stable record addresses, retained decisions, and completion receipts. The primary experiment comprises 528 structural cases, 928 fresh process attempts, and 256 abrupt exits, with separately committed coordinator and effect stores. Retained decisions produce 66 content passes per policy even without stable addresses, but only 37 selected terminal passes when receipts are absent. Stable addresses and retained decisions produce 66 terminal passes under either receipt setting; receipts reduce committed put/delete calls from 317 to 275 without changing planning counts. A native LangGraph implementation matches all seven measured outcomes in 66 corresponding cases. Extensions examine conflict contracts, two-process races, and operation lists from 24 fresh-context Sol 6.1 requests through Codex. These experiments separate historical effect recognition from preservation of current memory: an original-intent fact oracle can reward a stale overwrite. The contribution is an executable decomposition, bounded counterexamples, and auditable evidence built on established recovery techniques, rather than a new recovery algorithm or an evaluation of semantic memory quality.

1 Introduction

An agent decides to remember three preferences. Its first write commits, but the process stops before the remaining writes finish. On retry, should it choose a new plan, replay the previous plan, or return a recorded outcome? A fact-set check cannot fully answer this question. Repeating an insertion may duplicate a physical record without changing the fact set. Reusing an address may suppress that duplicate while writing a different value. Replaying the original value may restore the expected fact while destroying a newer update.

These questions matter because memory systems update and forget information across interactions [7, 23, 35]. They are also grounded in established systems research. RPC failure can leave execution uncertain; RIFL ties completion recognition to durable effects; Beldi replays logged choices [3, 14, 33]. Recent agent work evaluates ambiguous effects and recovery evidence across models and persistence hosts [15, 19]. Evaluation beyond recall is likewise established in governed-memory and lifecycle benchmarks [2, 30, 36].

Our question is narrower: which observations change when stable effect addresses (D), retained decisions and targets (P), and terminal receipts (R) vary separately under one explicit memory-mutation contract? The primary factorial uses a controlled decision tape to isolate execution from selection quality. It records content, physical cardinality, acknowledgement, decision continuity, planning invocations, and committed effects separately. Plan retention stays identical across receipt settings, so a receipt comparison does not silently change what decision remains recoverable.

The paper contributes three connected pieces of evidence: a complete eight-policy experiment over 66 specified history/profile cells per policy; a native checkpoint reference with the same bounded contract; and extensions that challenge what its success metric means under competing writes and recorded model replanning. The artifacts contain synthetic inputs, source, databases, process witnesses, raw model outputs, and separate evidence checkers. No proprietary application implementation or nonpublic conversation data is used. The purpose is a transparent conformance study and concrete counterexamples, rather than a claim that these recovery primitives are new.

*Independent research. Author background: AI Engineer at Genies; M.S. in Engineering, Purdue University.

2 Related Work

Memory and lifecycle evaluation. MemGPT, MemoryBank, and Mem0 develop persistent conversational memory, while A-MEM, Zep, Hindsight, and MemOS explore linked, temporal, retention-oriented, and operating-system abstractions [7, 13, 16, 23, 26, 32, 35]. LoCoMo and LongMemEval evaluate long-term conversational use; MemoryAgentBench evaluates incremental interactions; HaluMem studies hallucinations; UTILMEM studies utilization of distributed evidence [5, 11, 17, 25, 31]. Those semantic capabilities are not measured by equality of synthetic fact strings.

TimlMemBench separates canonical-pool answering from ingestion and governed-use cases such as revocation, supersession, scope, and provenance [36]. Prospective-memory evaluation distinguishes standing constraints and release authorities [2]. Memora evaluates consolidation and mutation, including obsolete information in responses [30]. Our execution-level observations complement these evaluations; we do not introduce evaluation beyond recall or show improved semantic memory quality. AgentPoison and MINJA address malicious memory contents under different attacker assumptions [6, 8]. Faithfully replaying a poisoned choice would not solve either threat.

Recovery and effect contracts. RIFL combines request identity, durable completion results, retry rendezvous, and safe reclamation; completion metadata must be atomic with its effects [14]. Beldi retains nondeterministic results and supports logged replay under its storage/API assumptions [33]. ARIES addresses database recovery, and sagas address application compensation for long activities [10, 21]. None of those guarantees follows merely from writing an agent plan to one file and its effects to another.

SagaLLM applies persistent context, validation, and compensation to agent planning. Atomix separates execution and settlement, while Agentic Transaction considers semantic transaction properties, including memory [4, 20, 29]. ACRFence evaluates restoration/replay and authority risks, distinguishing its implemented evaluation from a proposed analyzer [34]. LIMBO grades uncertain effects against service-side ledgers across models and harnesses; Permission to Retry separates recovery evidence, authority, completed work, and unresolved obligations on native hosts [15, 19]. These are close antecedents, not defeated baselines. Mansoor et al. combine postcondition verification, verify-before-retry, and action/payload-derived keys in a LangGraph agent evaluated on simulated non-atomic failures [18].

Systematic fault selection also predates this study: lineage-driven fault injection searches failures within a bounded model [1]. Our primary histories are manually specified rather than an exhaustive fault search. The contribution is the independently controlled D/P/R decomposition, inspectable observations, and boundary witnesses. Section 6 adds native persistence and model-output evidence while preserving that distinction.

3 Contract and Factorial Design

3.1 Intent, source, and decision

A logical intent j identifies one update; attempts a retry it. Every primary case has isolated stores and one fixed intent. The shared frontend selects enabled conversation rows with generation g and turn at most endpoint e, ordered by turn. The fixture uses $g = 0, e = 1$. Future and disabled rows make the source controls discriminating.

The controlled planner retains the latest selected fact per attribute and chooses an ordered operation list

$$P_j^{(a)} = \Pi(H_{e,g}, \rho, \xi_a) = (p_0, \ldots, p_{m-1}), \qquad p_i = (\mathrm{kind}_i, \mathrm{fact}_i, \mathrm{target}_i). \tag{1}$$

Here ρ is an operation profile and ξ_a a tape setting. Puts name an existing target or request insertion; deletes name a target. Changed-tape histories prescribe `topic=running` instead of `topic=cycling`. This is a selected continuity adversary, not a measured model error or natural drift rate. The primary planner neither calls an LLM nor infers targets from the evolving store.

3.2 Three controls

D: stable addresses. Existing named targets are bound under either setting. For targetless puts, D1 uses the first 24 hexadecimal characters of SHA-256 over intent and instruction slot (j, i); D0 generates a fresh 12-byte

random address at each application. D controls where a put writes, not which value it writes. Collision probability and identity allocation are outside this finite experiment.

P: retained decisions. P1 commits the literal ordered operations and target bindings before effects, then reloads that list on retry. P0 chooses again on reaching planning. Targetless operations remain targetless in the retained list: D0 generates fresh addresses even when P1 preserves the decision.

R: completion receipts. R1 checks a terminal receipt after the entry generation check, returning a recorded outcome on a hit. It stores the receipt after effect processing. R0 omits that lookup and storage. A P1 decision remains retained after success under *both* R settings. This fixed retention rule isolates receipt work suppression.

3.3 Separate observations

Let $\mathcal{R}$ be terminal physical records, $\phi(r)$ the fact in record r, E the expected fact set, and A a normal final process exit with an acknowledgement event. Define

$$C \equiv \{\phi(r) : r \in \mathcal{R}\} = E, \qquad K \equiv |\mathcal{R}| = |E|, \qquad T \equiv C \wedge K \wedge A. \tag{2}$$

C collapses identical duplicate facts; K observes physical row count. The selected terminal check T does not incorporate decision continuity. If $\mathcal{D}_j$ contains digests of decisions reported as chosen or loaded for intent j, continuity is

$$L \equiv \bigwedge_j (|\mathcal{D}_j| \leq 1). \tag{3}$$

A reported choice before durable preparation still counts for L. Planning invocations Q count decision-chosen events. Committed work W counts one audit row for each participant put/delete commit, including repeated deletion of an absent target. It excludes reset/cleanup SQL, reads, coordinator writes, tokens, latency, and monetary cost.

4 Implementation and Failure Model

The primary implementation uses Python's standard library and two SQLite files. The coordinator stores conversation, generation, decisions, and optional receipts. The effect store holds addressed records tagged by intent/generation, plus a commit audit. Each put/delete and audit entry commit together. The participant never reads the audit for deduplication. Preparation, effects, receipt finalization, reset, and cleanup have separate commits; no transaction spans the files. SQLite uses `synchronous=FULL` and `journal_mode=DELETE` [28].

Insert performs three targetless puts. Revise includes an existing-target unit update and two insertions. Retire deletes an obsolete target before three insertions. Mixed performs an existing-target unit put, a tone insertion, the named deletion, and a topic insertion. An exit after the first put need not be after the first effect.

Each fresh attempt executes the following order:

1. Compare current and expected generation. On mismatch, delete this intent's old-generation records, acknowledge cancellation, and return.
2. Under R1, return the recorded outcome if a matching receipt exists.
3. Under P1, load a retained decision; otherwise query eligible source and choose a list, saving it under P1. This is the preparation boundary.
4. Apply operations in order, committing each effect and audit row.
5. Recheck generation after the loop. On mismatch clean up old-generation intent records and select cancelled; otherwise select applied.
6. Under R1, commit the outcome receipt. Keep any P1 decision. Acknowledge and exit.

The harness terminates the child with `os._exit(87)` after preparation, the first put, all effects, or finalization, then retries in a new process [24]. Barriers follow returned commits; no SQLite commit is interrupted. Controlled unavailable-response failures raise an exception after a put/delete commit and exit with code 74.

Reset-before-retry advances generation after a first-put exit and removes fixture-origin rows, leaving intent residue for entry cleanup. Reset-between-effects uses a parent/child handshake: the child pauses after the first put, the parent advances generation and clears effects, then the child resumes. The post-loop check removes subsequent stale writes. This tests terminal cleanup, not absence of transient stale reads.

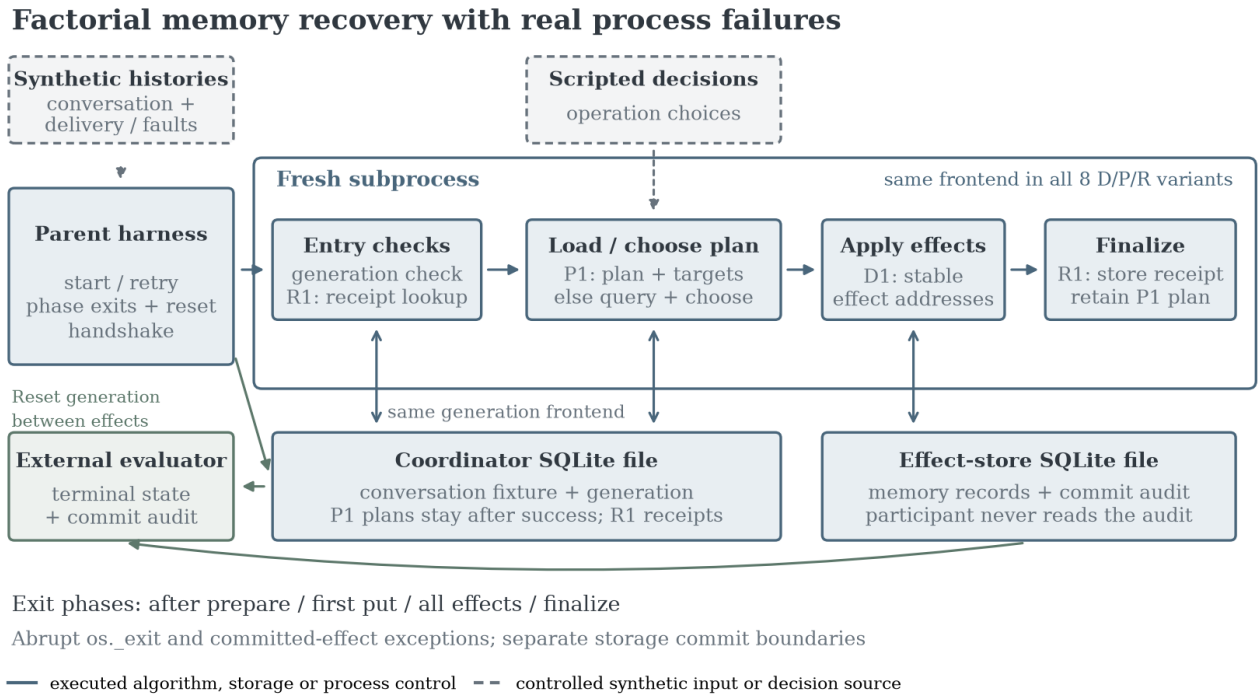


Figure 1: Primary factorial architecture. Controlled source, decision tape, and fault histories feed a common frontend and eight D/P/R policies. Fresh subprocesses reopen separate coordinator and effect stores. Generation checks precede receipt lookup and follow the effect loop; receipt/plan hits bypass source selection. The effect audit is evaluator-visible and not read by the participant. Dashed boxes are controlled inputs. The process frame does not imply cross-store atomicity.

5 Primary Results

5.1 Coverage and evidence

Seventeen histories, four profiles, and eight policies yield $17 \cdot 4 - 2 = 66$ cases per policy and 528 total: lost-delete-reply excludes two profiles without deletes. Histories include normal execution, fixed/changed completed redelivery, four exit barriers, unavailable put/delete replies, source controls, and two resets. Every case has distinct stores; renamed entities or reruns do not enlarge coverage.

Source and specification were frozen after development, before the final campaign. They were not externally preregistered or held out. Python 3.12.13/SQLite 3.53.1 produced 928 fresh attempts: 592 normal, 256 abrupt, and 80 controlled-exception exits. Abrupt exits comprise 64 after preparation, 96 after the first put, 64 after effects, and 32 after finalization. All final attempts acknowledge, so A does not distinguish main outcomes.

5.2 State, decision, and work

Table 1 gives the complete factorial. D0P0R0 passes content in 42 cases but T in only 29. P1 with D0R0 raises content to 66 while leaving 29 cardinality failures. D1P0R0 instead passes cardinality in all 66 but retains 24 content failures.

A first-put insertion exit illustrates the separation. The first attempt inserts `tone=concise`. D0P1R0 replays three facts to fresh addresses: content remains correct, but four rows exist. D1P0R0 with a changed tape reuses three addresses yet writes `topic=running`: cardinality passes while original-intent content and continuity fail. D1P1R0 preserves both by retaining the original tape and addresses.

Switching D adds 13, 9, 29, and 17 terminal passes in $(P, R) = (0,0), (0,1), (1,0), (1,1)$ order, without changing content, continuity, planning, or work counts. Switching P adds 24 content/continuity passes under R0 and 16

Table 1: Primary outcomes, 66 specified cases per policy. C, K, T, L count passes; Q, W count planning invocations and committed put/delete calls. Every policy acknowledges all 66 final attempts.

Policy	$C \uparrow$	$K \uparrow$	$T \uparrow$	$L \uparrow$	$Q \downarrow$	$W \downarrow$
D0P0R0	42	37	29	42	112	317
D0P0R1	50	49	41	50	100	275
D0P1R0	66	37	37	66	66	317
D0P1R1	66	49	49	66	66	275
D1P0R0	42	66	42	42	112	317
D1P0R1	50	66	50	50	100	275
D1P1R0	66	66	66	66	66	317
D1P1R1	66	66	66	66	66	275

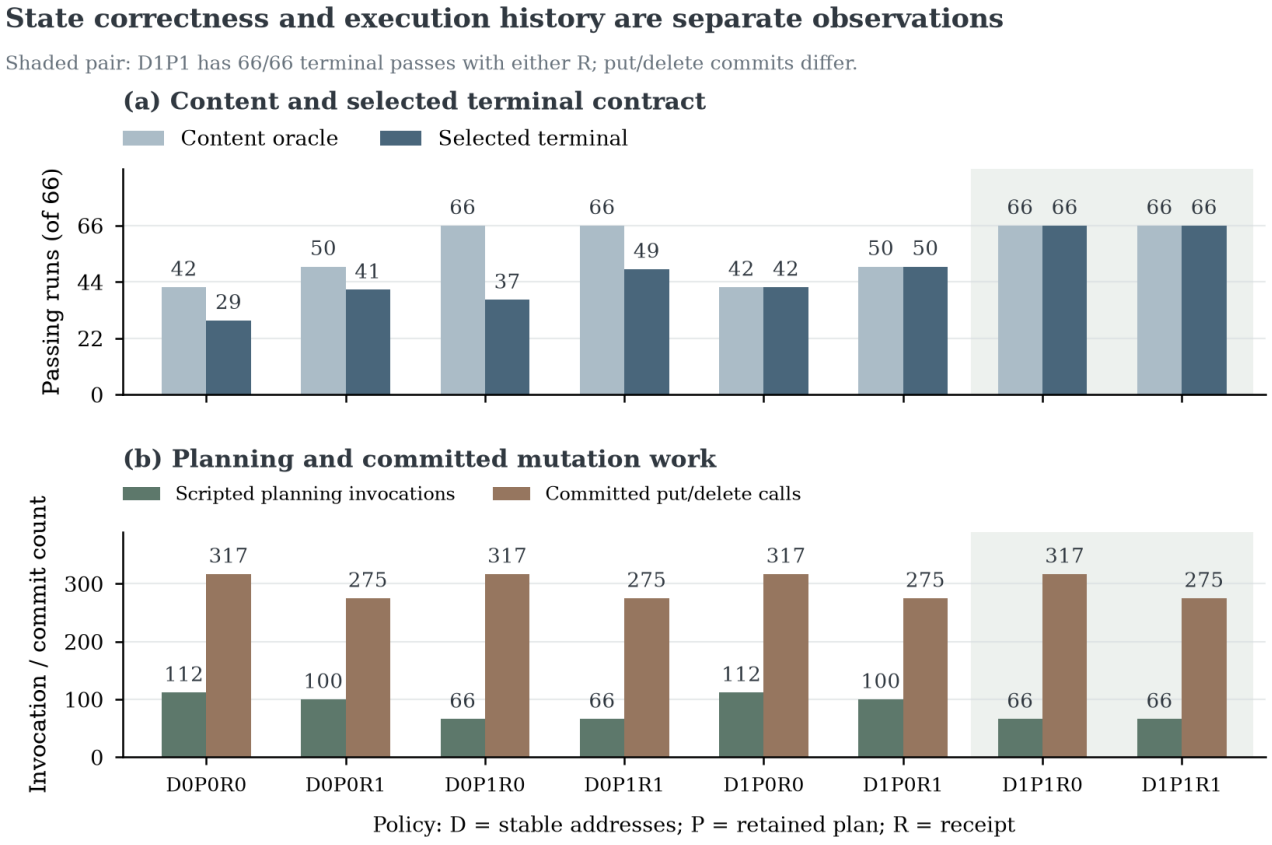


Figure 2: Primary content and terminal outcomes, with planning and committed-effect counts. Each policy has 66 structural cases. In the shaded D1P1 pair, both receipt settings pass T, but R1 suppresses 42 committed put/delete calls. Counts describe specified schedules, not population reliability or generic cost.

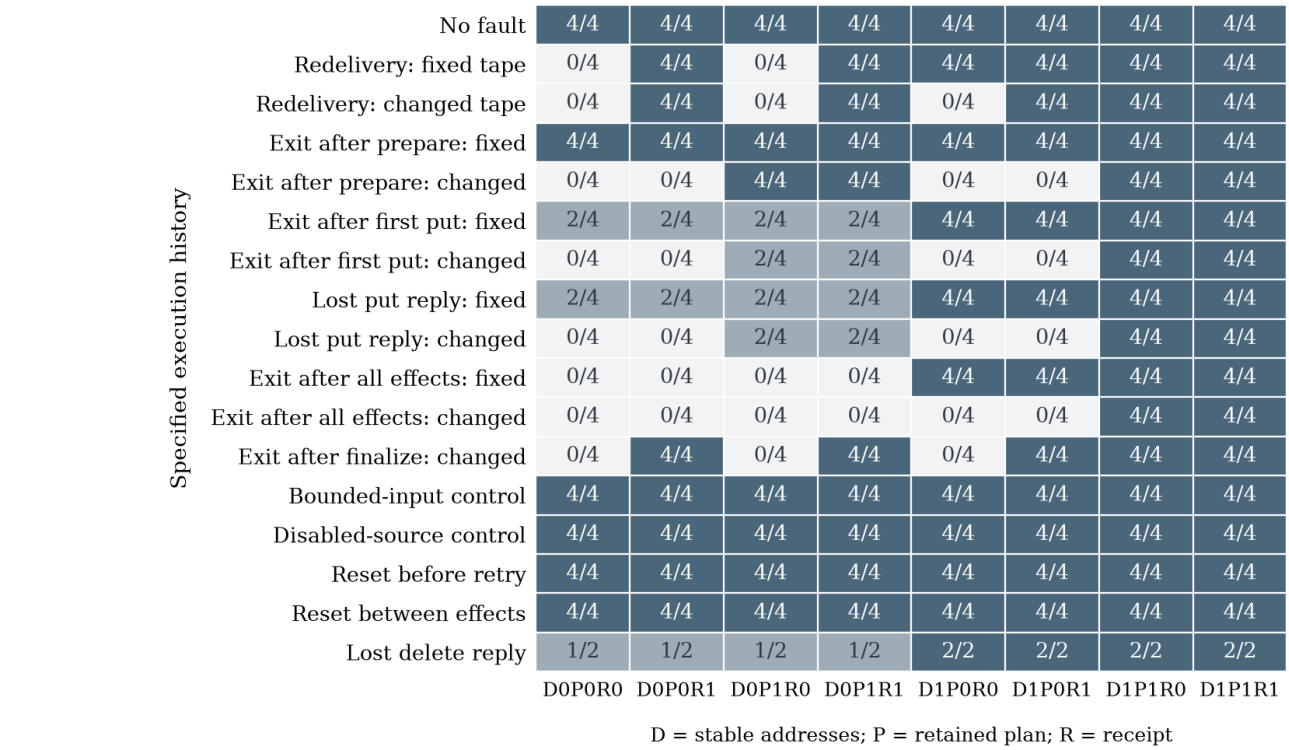


Figure 3: Terminal passes by declared history and policy. Cells show passing profiles/applicable profiles; lost-delete-reply has two profiles, all other histories four. Changed-tape cases prescribe a different later decision. Source and generation controls are shared, so their successes do not estimate a D/P/R gain.

under R1, saving 46 or 34 planning calls respectively without reducing effect calls. Its terminal gains are 8, 8, 24, and 16 in (D, R) order.

Switching R adds 12, 12, 8, and 0 terminal passes in (D, P) order. D1P1 passes all 66 T and L checks and plans 66 times under either R setting, but W falls from 317 to 275. A completed mixed-profile redelivery explains four saved calls: R0 replays three puts and a delete, whereas R1 returns the receipt. Receipt suppression and final-state conformance are distinct benefits.

5.3 Five boundary probes

Deleting only the receipt repeats the retained decision; deleting only the decision leaves the completed receipt fast path. Removing both permits a changed-tape replay. These are explicit deletion tests, not expiration policies. The before-store exit loses a reported but uncommitted choice, outside the prepared-decision obligation.

The last probe inserts a newer external value after the first put and before retry. Replaying the retained decision overwrites it with `unit=metric`. All original-intent checks pass despite losing that newer value. Thus T cannot certify latest-writer preservation; Section 6.4 tests that distinction under stronger store contracts.

Table 2: Separate D1P1R1 mixed-profile probes. All final attempts acknowledge. Q and W are counts, not pass metrics.

Probe	C	K	T	L	Q	W
Remove receipt only	pass	pass	pass	pass	1	8
Remove decision only	pass	pass	pass	pass	1	4
Remove both	fail	pass	fail	fail	2	8
Exit before decision store	fail	pass	fail	fail	2	4
Intervening external write	pass	pass	pass	pass	1	5

5.4 Checking and reproduction

A separate checker reads SQLite state and process witnesses without importing participant metric functions. It recomputes all observations and structural keys. A protocol audit checks targets, retained metadata, generation, and reconstructed state, including 688 primary source-selection observations. All 528 cases and five probes pass evidence-consistency checks. Ten focused tests cover actual exit/resume, oracle separation, fixed retention, source/generation controls, repeated deletes, changed decisions, and rejection of forged metrics.

A fresh same-host campaign with Python 3.14.3/SQLite 3.51.2 matches every primary scalar outcome and all five probes with unchanged source. Completed Ubuntu Python 3.12/3.14 runs also match frozen aggregates after raw-state verification. These are software-environment reproductions by the same study, not third-party replication or additional primary cases.

6 Reference and Boundary Extensions

6.1 Native checkpoint reference

A separate implementation uses LangGraph 1.2.18, checkpoint 4.3.1, and the SQLite checkpointer 3.1.2 [12]. A genuine `StateGraph` runs choose-decision → apply-effects → end with synchronous durability. Plans and completed results reside in native checkpoints; the primary coordinator's decision/receipt tables remain empty. On redelivery, application code inspects `graph.get_state`; an empty next-node list with a completed outcome takes an explicit return path without invoking the graph again. Application source/generation checks and the external effect ledger retain the same semantics.

All 66 primary D1P1R1 cells have corresponding native barriers. Preparation exits occur at apply-node entry after the decision checkpoint; prefix and post-effect faults occur inside that node before return. Finalization exits follow a returned completed invocation. Every attempt starts a fresh process and resumes from native state.

The reference matches all seven observations in every cell: C, K, T, L, A each pass 66, $Q = 66$, and $W = 275$. Its 116 attempts include 32 abrupt exits. A checker inspects 260 actual checkpoint rows, retained plans/results, effect state, and barrier witnesses. Nine focused tests pass. The single effects node deliberately matches the primary plan/apply decomposition; per-operation native tasks could change W and were not evaluated. This is portability evidence, not framework optimality or a comparison against a deployed memory system.

6.2 Recorded Sol 6.1 planning pilot

We froze a small protocol before requesting 24 fresh Codex sessions with `gpt-6.1-sol`, medium reasoning requested. Six synthetic conversations cover insertion, correction, retirement, mixed edits, a move, and a conditional preference. Each receives two repeats on the original store and two on a store after the first operation from original repeat 0. Both partial repeats see the same snapshot. Requests use an empty working directory, ephemeral session, ignored user configuration, disabled project documents, read-only permissions, and a fixed JSON schema [22].

All 24 responses satisfy the operation schema. Seven of 12 identical-prompt pairs differ in literal arguments even after ignoring JSON key order and operation-list order; examples include `reply_style` versus `reply_length` and synonymous wording. All 12 original-to-partial comparisons differ. The latter compare changed inputs, not same-input variability. A separate checker verifies prompts, raw responses, the partial-snapshot rule, and counts.

Literal variation does not establish semantic disagreement or harmful drift. The pilot has one requested alias, one harness, six authored conversations, and two repeats per condition. Exact backend snapshot, temperature, seed, and system prompt are not controlled or exposed by the retained CLI events. We report observations, not a drift-rate estimate, accuracy score, or statistical significance test.

6.3 Plan compaction and positional identity

After examining the recorded outputs, we froze an exploratory integration protocol: 12 original-repeat-0/partial-repeat plan pairs, each replayed in three SQLite arms. All arms start from the exact archived partial snapshot with a common bijective public-ID/physical-address binding. The arms use (i) the recorded partial plan with stable intent/slot addresses, (ii) that same plan with fresh insertion addresses, or (iii) the retained original plan with its original stable addresses. Named targets are held fixed.

Two positional-replan integrations overwrite a completed prefix record with a different fact key; the other ten preserve the prefix. Fresh-address and retained-plan controls preserve it in all 12 pairs each. Both overwrites are the two partial responses of the same preferences scenario, the only scenario with a targetless first operation. They are not two independently sampled failure types or an estimated model failure rate.

The witness requires no contradictory model choice. Original slot 0 stores `reply_style`; the partial snapshot already contains it. The model omits that completed fact and begins its remaining plan with `preferred_units`. The adapter derives the same physical slot-0 address and overwrites the style record. Giving the identical remaining plan fresh addresses prevents that overwrite; retaining the original ordered plan preserves the original identity-to-operation binding. All 36 integrations, their before/after audit rows, and three focused control tests are retained. No additional model requests or semantic-synonym grading are involved.

This is an adapter contract failure, not evidence of a LangGraph defect or unsafe idempotency keys in general. LangGraph already warns that changing task/interrupt order can mismatch cached values [12]. Featonby describes retaining original parameters and rejecting changed intent under the same request identifier [9]; our positional upsert has no such rejection. The concrete model-output witness illustrates that known identity/order hazard when a memory planner compacts a plan by omitting an already represented fact. Stable logical-operation identities or mismatch rejection are additional controls, not evaluated remedies here.

6.4 Conflict recognition and stale overwrites

An exploratory store-contract extension compares unconditional upsert, value comparison, monotonic-version comparison, and version comparison with an atomic per-operation receipt. A successful mutation and its receipt commit in the *same* SQLite transaction in the last contract, unlike the primary cross-store terminal receipt. In that receipt-enabled contract, once a successful operation has a retained receipt, reusing its identity with changed arguments is rejected; a failed comparison does not bind that identity. Deletion retains a versioned tombstone.

The bounded campaign contains 48 sequential schedules (six first-before-retry linearizations of two intents, two put/delete profiles, four contracts), 20 named witnesses, and 16 two-process races. Both processes rendezvous before writing; SQLite serializes transactions, and the winner is uncontrolled. Reply-loss witnesses model reissued calls after a commit; no transport packet is dropped. The checker reconstructs state from the raw effect journal without importing store code. All 84 cases pass declared evidence checks; eight focused contract tests pass.

Three witnesses identify different obligations. First, an exact replay after its own committed write receives a version conflict under version comparison alone, but historical-success acknowledgement under the per-effect receipt. Second, after A commits, B commits a newer value, and A retries, unconditional upsert restores A and passes its original-intent content check while erasing B. The versioned receipt recognizes A's historical effect without changing B. Third, value-only comparison accepts a stale operation after a value changes and returns to its old value (ABA); monotonic versions reject it. The delete/recreate witness retains this distinction through a tombstone.

These apply established idempotency and concurrency ideas, including RIFL's distinction between executing a request once and preserving its result against subsequent writers [14]. They are not a new transaction protocol. For a single-valued target, if A's desired value is x and a later accepted writer establishes $y \neq x$, then

$$C_A \Rightarrow v = x, \qquad F_B \Rightarrow v = y, \qquad C_A \wedge F_B = \text{false}. \tag{4}$$

Here F_B means preserving B absent an accepted later update. Historical acknowledgement of A can be valid while C_A is false. Evaluation must specify which obligation takes precedence.

Under retained immutable receipts and monotonic versions, the narrow per-effect invariant follows directly: an exact successful replay finds a receipt and does not mutate; an unseen stale operation fails its version check. Neither branch overwrites a newer version. The argument assumes atomic receipt/effect storage, fixed identity/arguments, and retained version history. It proves neither whole-plan atomicity nor safe reclamation and does not transfer to the primary two-file receipt scheme.

7 Discussion and Validity

The evidence distinguishes record address, chosen operation, historical effect, and current fact. Stable addresses repair duplicate physical insertion only under a stable binding between identity and operation. Retained choices preserve a decision, including an incorrect decision. Receipts suppress repeated work or recognize historical success; they do not keep current memory equal to an earlier intent's desired state forever.

The primary counts are exact properties of authored cases, not estimates from a user population. The native reference is one framework with application-specific guards. The pilot supplies actual model plans but no semantic correctness labels, model comparison, or natural workload distribution. The integration and conflict extensions were motivated by review and observed outputs; they are exploratory, not held-out confirmation. Same-author checking, code review, and CI reduce implementation errors but do not replace outside replication.

All effects occur in local SQLite under specified API semantics. Primary exits follow semantic barriers rather than interrupting storage commits. The two-process extension bounds contention and several linearizations; it does not enumerate arbitrary distributed interleavings. Power loss, corruption, remote late commits, transient-reader isolation, cross-user identity allocation, unbounded retention, and an adversarial authority model remain untested. No recall, answer quality, poisoning defense, or production reliability follows from these results.

A broader research claim would require multiple independently implemented memory systems, realistic workloads, explicit concurrency/retention contracts, larger controlled model experiments with semantic adjudication, and outside replication. This artifact offers a reproducible starting point and falsifiable contract distinctions for that work.

8 Conclusion

Fact sets, physical records, retained decisions, and effect histories answer different recovery questions. The 528-case factorial separates their dependencies; a 66-case native reference reproduces the measured contract. Recorded Sol 6.1 plans and bounded conflict tests then challenge what a terminal pass means when a plan changes or another writer acts. Evaluating agent memory requires separate operation-identity, historical-completion, and current-state obligations, with tests at their boundaries.

References

[1] Peter Alvaro, Joshua Rosen, and Joseph M. Hellerstein. Lineage-driven Fault Injection. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pages 331–346. ACM, 2015. doi: 10.1145/2723372.2723711. URL https://people.ucsc.edu/~palvaro/molly.pdf.

[2] Deniz Bilgin, Jonas Schewior, and Andreas Wendemuth. Capture is easy, forgetting is hard: Benchmarking prospective memory in conversational agents. In *Proceedings of the Second Workshop for REsearch on Agent Language Models (REALM 2026)*, 2026. URL https://aclanthology.org/2026.realm-1.43/.

[3] Andrew D. Birrell and Bruce Jay Nelson. Implementing Remote Procedure Calls. *ACM Transactions on Computer Systems*, 2(1):39–59, 1984. doi: 10.1145/2080.357392. URL https://www.birrell.org/andrew/papers/.

[4] Edward Y. Chang and Longling Geng. Sagallm: Context management, validation, and transaction guarantees for multi-agent llm planning. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2503.11951.

[5] Ding Chen, Simin Niu, Kehang Li, Peng Liu, Xiangping Zheng, Bo Tang, Xinchi Li, Feiyu Xiong, and Zhiyu Li. Halumem: Evaluating hallucinations in memory systems of agents. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2511.03506.

[6] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. In *Advances in Neural Information Processing Systems 37*, pages 130185–130213. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024. doi: 10.52202/079017-4136. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/eb113910e9c3f6242541c1652e30dfd6-Abstract-Conference.html.

[7] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2504.19413.

[8] Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen J. Xiang. Memory Injection Attacks on LLM Agents via Query-Only Interaction. In *Advances in Neural Information Processing Systems 38*, pages 52119–52153. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2025. doi: 10.52202/085713-1554. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/42a97bbd9844d2bf68596730af80bcdf-Abstract-Conference.html.

[9] Malcolm Featonby. Making retries safe with idempotent APIs. The Amazon Builders' Library, n.d. URL https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/. Section: Same client request ID, different intent. Accessed October 29, 2026.

[10] Hector Garcia-Molina and Kenneth Salem. Sagas. In *Proceedings of the 1987 ACM SIGMOD international conference on Management of data - SIGMOD '87*, pages 249–259. ACM Press, 1987. doi: 10.1145/38713.38742. URL https://www.cs.princeton.edu/research/techreps/598. Primary text inspected in Princeton technical report TR-070-87, January 1987.

[11] Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in llm agents via incremental multi-turn interactions. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2507.05257.

[12] LangChain. Graph API overview. LangGraph documentation, n.d. URL https://docs.langchain.com/oss/python/langgraph/graph-api. Section: Re-execution and idempotency. Accessed October 29, 2026.

[13] Chris Latimer, Nicoló Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight is 20/20: Building agent memory that retains, recalls, and reflects. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2512.12818.

[14] Collin Lee, Seo Jin Park, Ankita Kejriwal, Satoshi Matsushita, and John Ousterhout. Implementing Linearizability at Large Scale and Low Latency. In *Proceedings of the 25th Symposium on Operating Systems Principles*, pages 71–86. ACM, 2015. doi: 10.1145/2815400.2815416. URL https://seojinpark.net/projects/rifl/.

[15] Jiapeng Li. Where does exactly-once live? model, harness, and tool-contract effects on duplicate side effects in llm agents. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2609.29095.

[16] Zhiyu Li, Shichao Song, Hanyu Wang, Simin Niu, Ding Chen, Jiawei Yang, Chenyang Xi, Huayi Lai, Jihao Zhao, Yezhaohui Wang, Junpeng Ren, Zehao Lin, Jiahao Huo, Tianyi Chen, Kai Chen, Kehang Li, Zhiqiang Yin, Qingchen Yu, Bo Tang, Hongkang Yang, Zhi-Qin John Xu, and Feiyu Xiong. Memos: An operating system for memory-augmented generation (mag) in large language models. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2505.22101.

[17] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024. doi: 10.18653/v1/2024.acl-long.747. URL https://aclanthology.org/2024.acl-long.747/.

[18] Isham Kalappurackal Mansoor, Abhishek Phadke, and Pratip Rana. Verified tool calls improve LLM agent reliability under non-atomic failures. *arXiv preprint arXiv:2608.02645*, 2026. doi: 10.48550/arXiv.2608.02645. URL https://arxiv.org/abs/2608.02645.

[19] Deekshith Marla and Kushagra Bhatnagar. Permission to retry: A study of authorization and recovery in ai agent workflows. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2610.31916.

[20] Bardia Mohammadi, Nearchos Potamitis, Lars Klein, Akhil Arora, and Laurent Bindschaedler. Atomix: Timely, transactional tool use for reliable agentic workflows. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2602.14849.

[21] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992. doi: 10.1145/128765.128770. URL https://doi.org/10.1145/128765.128770.

[22] OpenAI. Non-interactive mode. Codex documentation, n.d. URL https://learn.chatgpt.com/docs/non-interactive-mode. Accessed October 29, 2026.

[23] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint*, 2023. URL https://arxiv.org/abs/2310.08560.

[24] Python Software Foundation. os._exit: process termination in Python 3.12, 2026. URL https://docs.python.org/3.12/library/os.html#os._exit. Official documentation; publication date unspecified; accessed 2026-10-29.

[25] Peijun Qing, Fobo Shi, and Soroush Vosoughi. Utilmem: Benchmarking evidence utilization in long-term conversational memory. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, 2026. URL https://aclanthology.org/2026.emnlp-main.1639/.

[26] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2501.13956.

[27] Yiwen Song, Yale Song, Tomas Pfister, and Jinsung Yoon. Paperorchestra: A multi-agent framework for automated ai research paper writing. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2604.05018.

[28] SQLite. Atomic Commit In SQLite, 2026. URL https://www.sqlite.org/atomiccommit.html. Official documentation; publication date unspecified; accessed 2026-10-29.

[29] Zhaoyan Sun, Xiaoxiao Wang, and Guoliang Li. Agentic transaction: Towards acid-compliant agent systems. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2608.13900.

[30] Md Nayem Uddin, Kumar Shubham, Eduardo Blanco, Chitta Baral, and Gengyu Wang. From Recall to Forgetting: Benchmarking Long-Term Memory for Personalized Agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 26814–26841. Association for Computational Linguistics, July 2026. doi: 10.18653/v1/2026.findings-acl.1337. URL https://aclanthology.org/2026.findings-acl.1337/.

[31] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2410.10813.

[32] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2502.12110.

[33] Haoran Zhang, Adney Cardoza, Peter Baile Chen, Sebastian Angel, and Vincent Liu. Fault-tolerant and transactional stateful serverless workflows. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1187–1204. USENIX Association, 2020. URL https://www.usenix.org/conference/osdi20/presentation/zhang-haoran.

[34] Yusheng Zheng, Yiwei Yang, Wei Zhang, and Andi Quinn. Acrfence: Preventing semantic rollback attacks in agent checkpoint-restore. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2603.20625.

[35] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. *arXiv preprint*, 2023. URL https://arxiv.org/abs/2305.10250.

[36] Fangxing Zhou, Yifei Sun, Jiachen Li, Jinyu Xu, Yuhan Wang, Ruixin Li, and Qing Xie. Timlmembench: A diagnostic benchmark for governed use of long-term memory in dialogue agents. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, 2026. URL https://aclanthology.org/2026.findings-emnlp.507/.

A Artifacts, Reproduction, and Assistance

The public companion repository is `https://github.com/aditya-genies/beyond-recall`. It contains synthetic inputs, frozen primary data, databases, checkers, figure sources, extension protocols, model prompts/responses, and dependency pins. Source/data manifests identify each campaign separately; counts are not pooled.

```
python3 -W error::ResourceWarning -m unittest -v test_study
python3 study.py campaign --out my-results --workers 4
python3 checks/check_results.py --campaign my-results \
  --out my-verification.json
python3 compare_results.py --candidate my-results
python3 extensions/sol_pilot/check.py \
  --out extensions/sol_pilot/registered
python3 extensions/conflict_contracts/check.py \
  --out extensions/conflict_contracts/results
```

Native-framework and integration instructions are in their extension directories. The model runner requires an authenticated personal Codex account and has no API-key fallback. A fresh model request need not reproduce its literal output; archived responses make downstream integration rerunnable without model access.

The primary source digest remains

```
98378f217f0dae050dbc07aa04e1121ed9bc73862c2ee35a2e31a7a464f4e9b9
```

The primary cases digest remains

```
6714681fbefcbdf63334f76b95410aa1c2bc620ce8b7511fadb0c91354b72d01
```

Full SHA-256 manifests cover remaining evidence. New experiments have separate specifications and do not overwrite the primary study.

PaperOrchestra guided outline, literature, figures, drafting, and refinement [27]. AI assistance also supported implementation, tests, checks, and prose. The labelled Sol 6.1 pilot is experimental data, separate from authoring assistance. The author is responsible for claims and release decisions. This is a working manuscript; public availability is not peer-reviewed publication or an acceptance claim.